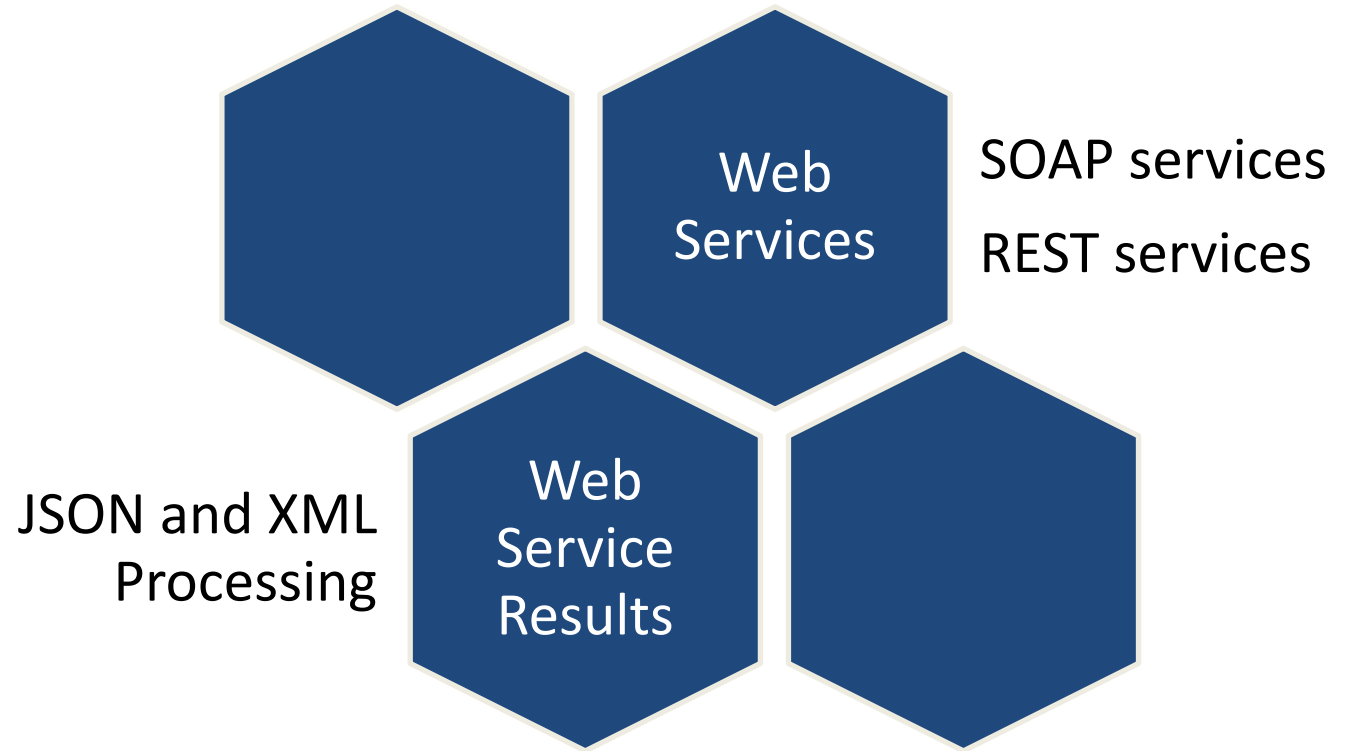




Mobile Devices and Applications

Unit 8 – SOAP & REST Services / XML
& JSON

Summary





WEB SERVICES



SOAP Services



kSOAP2 library

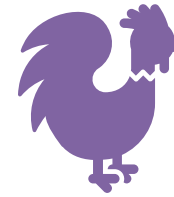


Classes:

SoapObject, object used for setting the method's parameters;

SoapSerializationEnvelope, object used for enveloping the connection attributes

HttpTransportSE, object used for making the connection to the web service;



Methods:

addProperty, add a parameter for the service method;

setOutputSoapObject, sets the object in which the response will be written;

call, makes a call to a specified service method

Service description - WSDL

- **WSDL** is an XML format for describing network **services** as a set of endpoints;
- The endpoints are operating on **messages**;
- The operations and messages are described abstractly;
- A defined endpoint represents a set of messages and operations bound to a concrete network protocol and message format;
- Example:
<http://www.w3schools.com/xml/tempconvert.aspx?WSDL>

Soap Request

```
POST /xml/tempconvert.aspx HTTP/1.1
Host: www.w3schools.com
Content-Type: text/xml; charset=utf-8
Content-Length: length
SOAPAction: "http://www.w3schools.com/xml/FahrenheitToCelsius"
<?xml version="1.0" encoding="utf-8"?>
<soap:Envelope xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:xsd="http://www.w3.org/2001/XMLSchema"
xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <FahrenheitToCelsius xmlns="http://www.w3schools.com/xml/">
      <Fahrenheit>string</Fahrenheit>
    </FahrenheitToCelsius>
  </soap:Body>
</soap:Envelope>
```

Soap Response

```
HTTP/1.1 200 OK Content-Type: text/xml; charset=utf-8 Content-Length: length <?xml
version="1.0" encoding="utf-8"?>
<soap:Envelope xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:xsd="http://www.w3.org/2001/XMLSchema"
xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
<soap:Body>
    <FahrenheitToCelsiusResponse xmlns="http://www.w3schools.com/xml/">
        <FahrenheitToCelsiusResult>string</FahrenheitToCelsiusResult>
    </FahrenheitToCelsiusResponse>
</soap:Body>
</soap:Envelope>
```

Web REST

Actions are implemented by using the HTTP protocol

List of possible operations:

GET, POST, PUT, DELETE, etc.

Results:

JSON (JavaScript Object Notation)

XML (eXtended Markup Language)



XML AND JSON FILES



XML Files

- Elements seen as nodes;
- A root node representing the element or a collection of elements;
- Attributes with their values seen as child nodes of an element;
- Used to store data and data structure;

```
<?xml version="1.0" encoding="ISO-8859-1"?>
```

```
<root_element>
```

```
  <element attribute="attribute_value">  
    element value or other elements
```

```
  </element>
```

```
  ...
```

```
</ root_element >
```

XML File Example

XML Files

```
<?xml version="1.0" encoding="ISO-8859-1"?>
```

```
<library>
```

```
  <book id="19222">
```

```
    <author>
```

```
      <name>R. Meier</name>
```

```
    </author>
```

```
    <title> Professional Android Application  
Development </title>
```

```
    <publisher>Wiley</ publisher>>
```

```
    <year>2009</year>
```

```
    <isbn>1-72-11-2222</isbn>
```

```
    <pages>500</pages>
```

```
  </ book >
```

```
< book id="19223">
```

```
  <author>
```

```
    <name>S. Hashimi</name>
```

```
    <name> S. Komatineni</name>
```

```
    <name> D. MacLean </name>
```

```
  </author>
```

```
  <title>Pro Android 3</title>
```

```
  <publisher>Apress</publisher>
```

```
  <year>2011</year>
```

```
  <isbn>0-321-15040-6</isbn>
```

```
  <pages>336</pages>
```

```
</book >
```

```
</ library >
```



PARSING XML AND JSON FILES

XML Files

- SAX (Simple API for XML)
 - org.xml.sax
 - *SAXParserFactory, SAXParser*
 - Sequential reading of a XML document
 - Inverse order events
- XML Pull
 - org.xmlpull.v1
 - *XmlPullParserFactory, XmlPullParser*
 - Sequential reading of a XML document
 - Directly treated events
- DOM
 - org.w3c.dom
 - *DocumentBuilderFactory, DocumentBuilder, Document*
 - An hierarchical structure is generated and loaded in memory
 - The nodes are managed in lists of nodes

SAX

- **SAXParserFactory**
 - Used for creating **SAXParser** objects
- **SAXParser**
 - Used for processing the XML files
- **XMLReader**
 - Used to read the file
 - **parse()**
- **Methods:** `getParser()`, `getSchema()`, `getProperty()`
- **DefaultHandler:** receives callbacks when notifications are triggered

DefaultHandler

Methods called when processing the XML file

startElement()

- Called when start reading an element;

endElement()

- Called when the element was completely read;

characters()

- Called when a character sequence appears;

Is hooked up to the XmlReader

- **setContentHandler()**

Returns the result

SAX Example

```
SAXParserFactory fact =  
SAXParserFactory.newInstance();
```

```
SAXParser xmlParser = fact.newSAXParser();
```

```
XMLReader xmlReader =  
xmlParser.getXMLReader();
```

```
SAXHandler handler = new SAXHandler();
```

```
xmlReader.setContentHandler(handler);
```

```
xmlReader.parse(new InputSource(streamIn));
```

XML Pull

- XmlPullParser interface
- Initialization
 - Resources#**getXml()**
 - res/xml
 - Xml.**newPullParser()**
 - XmlPullParserFactory#**newPullParser()**
 - XmlPullParserFactory
 - XmlPullParserFactory.**newInstance()**
- Needs an associated input stream (InputStream)
 - setInput()

XmlPullParser

- **next()** – parsing the document
- Events:
 - Types: **START_TAG/END_TAG, TEXT, START_DOCUMENT/END_DOCUMENT**
 - **getEventType()**
- **nextToken()** – advances with the reading process
- **getName()** – gets the tag name
- **getText()** – gets the content of a tag
- **getAttributeCount()** – gets the attribute number
- **getAttributeValue()** – gets the attribute value

XmlPullParser Example

```
static ArrayList<Book> processXML_Pull(InputStream isXML) {  
    ArrayList<Book> list = new ArrayList<Book>();  
    Book book = null;  
    int event; String text = null;  
    try {  
        // creates the parser  
        XmlPullParser xmlParser = Xml.newPullParser();  
        xmlParser.setInput(isXML, null);  
        event = xmlParser.getEventType();  
        while (event != XmlPullParser.END_DOCUMENT) {  
            String name = xmlParser.getName();  
            //the switch from the next slide  
            event = xmlParser.next();  
        }  
    } catch (Exception e) {e.printStackTrace(); }  
    return list;}  
}
```

XmlPullParser Example

```
switch (event) {  
case XmlPullParser.START_TAG:  
    if (name.equals("book")) {  
        book = new Book();  
        book.setId(xmlParser.getAttributeValue(null,"id"));  
    }  
    break;  
case XmlPullParser.TEXT:  
    text = xmlParser.getText();  
    break;  
case XmlPullParser.END_TAG:  
    if (name.equals("title")) {  
        book.setTitle(text);  
    }  
    if (name.equals("book")) {  
        list.add(book);  
    }  
    break;  
}
```

DOM

Interfaces

- **Node**
- **Element** (impl. Node)
- **NodeList**
 - **item(position)**

DocumentBuilder class

- **parse()** method
 - **InputStream**

Document (impl. Node) class

DOM

- **Document**

- **getElementsByTagName()**
 - NodeList

- **Element**

- **getAttribute()**

- **Node**

- **getTextContent()**

DOM Example

```
DocumentBuilderFactory docBuilderFactory =  
    DocumentBuilderFactory.newInstance();
```

```
DocumentBuilder docBuilder =  
    docBuilderFactory.newDocumentBuilder();
```

```
//gets the input stream
```

```
InputStream is = getResources().openRawResource(R.raw.biblx);
```

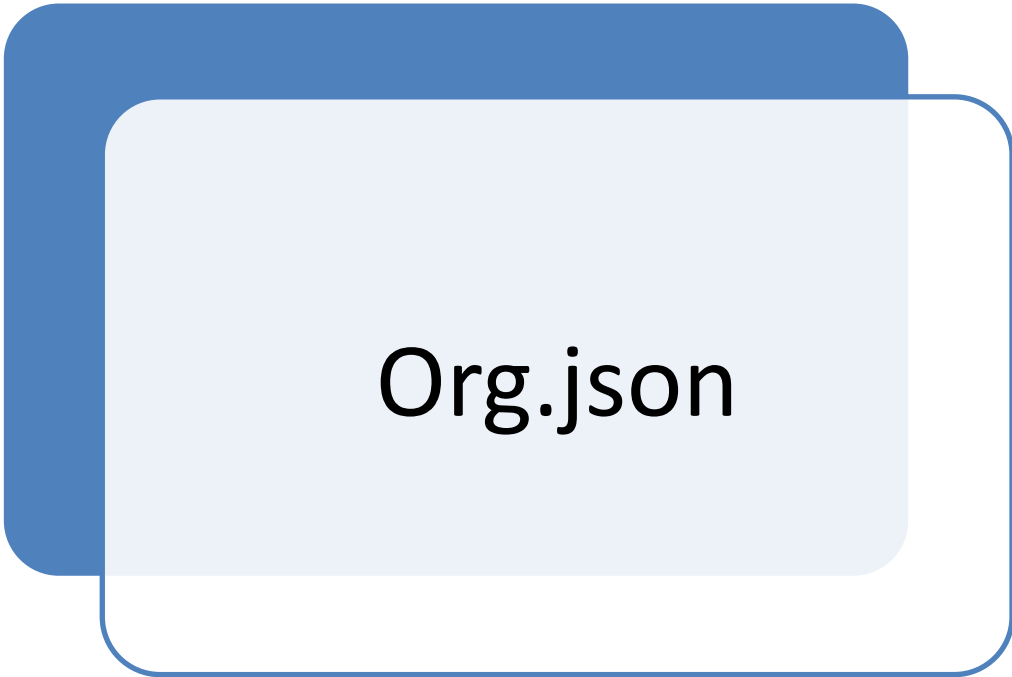
```
//create the document
```

```
Document docXml = docBuilder.parse(is);
```

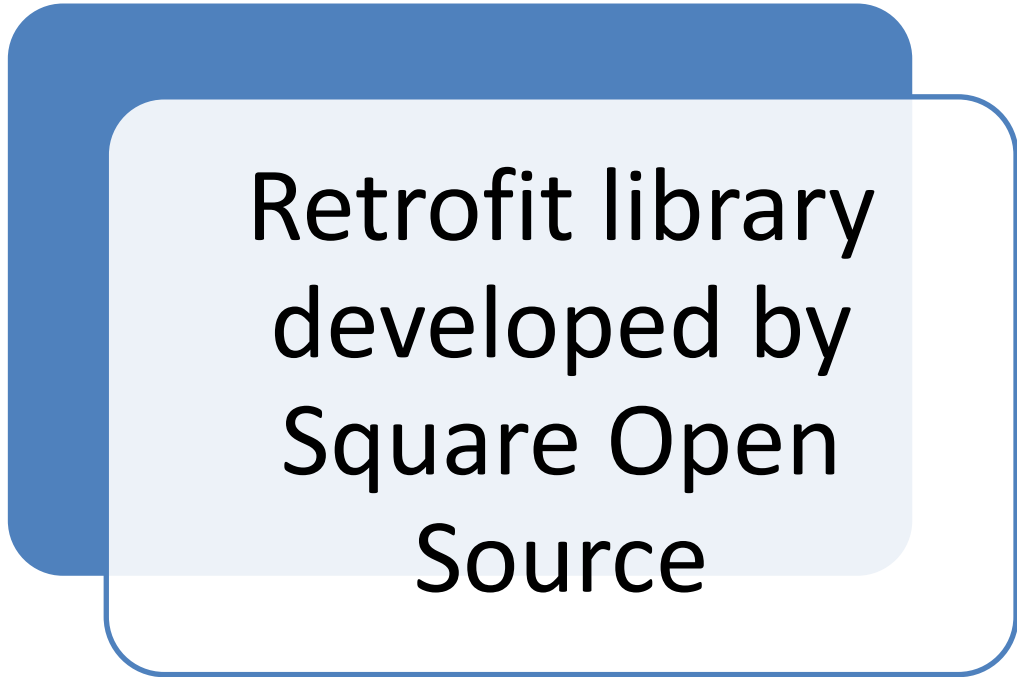
```
if (docXml != null) {  
    //gets the list of nodes  
    NodeList books = docXml.getElementsByTagName("book");  
    //read each node from the list  
    for (int i = 0; i < books.getLength(); i++) {  
        //book processing  
        Node nodCrt = books.item(i);  
  
        if (nodCrt.getNodeType() == Node.ELEMENT_NODE) {  
            Element obj= (Element) nodCrt;  
            Book book = new Book();  
            book.id = obj.getAttribute("id");  
            book.title = obj.getElementsByTagName("title").  
                item(0).getTextContent();  
            book.isbn = obj.getElementsByTagName("isbn").  
                item(0).getTextContent();  
        }  
    }  
}
```



JSON Files



Org.json



Retrofit library
developed by
Square Open
Source

JSON Files

an independent data exchange format;

designed for representing simple structures;

limited to text and numeric values;

`{ }` – describes the object

`[ ]` – represents a list of values

`"property" : "value"`

JSON Files

```
{
  "library":
  {
    "book":
    [
      {
        "-id": "19222", "author": { "name": "R. Meier" }, "title": " Professional Android Application
        Development ", "publisher": "wiley", "year": "2009", "isbn": "1-72-11-2222", "pages": "500"
      },
      {
        "-id": "19223", "author": { "name": [ "S. Hashimi", " S. Komatineni", " D. MacLean " ] }, "title":
        "Pro Android 3", "publisher": "Apress", "year": "2011", "isbn": "0-321-15040-6", "pages": "336"
      }
    ]
  }
}
```

JSON Files

- org.json
- JSONObject
 - *getType()*: *getString()*, *getBoolean()*, *getInt()* etc.
 - *getJSONArray()*
 - *getJSONObject()*
- JSONArray
 - *length()*
 - *getTIP(index)*

JSON Files Example

```
JSONObject jobject = new JSONObject(stringJson);
JSONObject joLibrary = jobject.getJSONObject("library");
JSONArray jaBooks = joLibrary.getJSONArray("book");

for (int i = 0; i < jaBooks.length(); i++) {
    Book book = new Book();
    //book processing
    JSONObject joBook = jaBooks.getJSONObject(i);
    String title = joBook .getString("title");
    book.setTitle(title);
    int year = joBook.getInt("year");
    String id = joBook.getString("-id");
    ....
}
```

Retrofit

POJO objects (Plain Old Java Object) – the model class

Interface for managing the URL's – the service class

RestAdapter – the rest client class

By default Retrofit uses Gson but other converters can be also used;

POJO class

- <http://www.jsonschema2pojo.org>
- <http://pojo.sodhanalibrary.com>
- Used for creating the Java class associated with the JSON or XML objects

Retrofit Interface

- Manages the URLs to each the service class must connect;
- Manages the URL calls like GET, PUT, etc.

```
@GET("/data/2.5/weather")  
public void getWeather(@Query("q") String  
city, @Query("APPID") String apikey,  
Callback<Model> response);
```

RestAdapter

```
RestAdapter restAdapter = new  
RestAdapter.Builder().setEndpoint(API)  
.build();
```

```
RestInterface restInterface =  
restAdapter.create(RestInterface.class  
);
```

Resources

- C. Boja, C. Ciurea, M. Doinea – Android Mobile Applications: A Practical Development Guide, ASE Press, 2015, 404 pp., ISBN 978-606-34-0033-9
- S. Komatineni, D. MacLean – Pro Android 4, Apress, 2012
- R. Meier – Professional Android 4 Application Development, Wiley, 2012
- P. Pocatilu, Programarea dispozitivelor mobile, Editura ASE, 2012
- P. Pocatilu, I. Ivan ș.a. – Programarea aplicațiilor Android, Editura, ASE, 2015
- <http://developer.android.com>